

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192521391
Name	BRIJESH.T

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I BRIJESH.T/192521391 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: _____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. HDFS-Based Storage Solution for a Media Platform

For a media platform, large files such as videos, images, and audio need to be stored reliably. I would use **HDFS with a replication factor of 3**.

Design:

Media Files → HDFS → DataNodes

↓

NameNode + Standby NameNode

- **HDFS** is suitable because it can store very large media files across many servers.
- Set the **replication factor to 3**, which is the maximum allowed.
- Each block is stored on three different DataNodes so that data is still available if one or two nodes fail.
- Use an **Active NameNode and Standby NameNode** for high availability.
- Use **large HDFS blocks** to reduce metadata overhead for large video files.
- Add DataNodes when storage requirements increase. This provides **horizontal scalability**.
- Old or less-used media can be moved to cheaper storage to reduce cost.
- Data compression can be used for suitable files.
- Monitor DataNodes and disk usage regularly.

Justification:

Replication factor 3 provides good availability without exceeding the given limit. HDFS distributes data across multiple nodes, so the platform can continue working during individual node failures. Adding storage nodes when required keeps the system cost efficient.

2. MapReduce Workflow for a 20-Minute Batch Window

The main requirement is to complete log processing within **20 minutes using limited cluster nodes**.

Job design:

Log Files → Input Splitting → Map Tasks → Shuffle & Sort → Reduce Tasks → Output

Map Phase

- Divide large log files into blocks.
- Each block is processed by a separate **Mapper**.
- The Mapper extracts useful information such as IP address, URL, status code, and access count.
- Mappers should run close to the data to reduce network traffic.

Shuffle and Sort

- Intermediate results are grouped according to keys.
- Use a **Combiner** where possible to reduce the amount of data transferred to Reducers.

Reduce Phase

- Reducers combine the intermediate results.
- They generate final results such as total requests, errors, popular pages, or traffic by IP.

Task Distribution

- Assign Map tasks across the available nodes.
- Give more tasks to nodes with better available resources.
- Use data locality so that processing happens near the stored log data.
- Avoid too many Reducers because the cluster has limited nodes.
- Monitor slow tasks and use speculative execution if necessary.

Justification:

Using parallel Map tasks, combiners, proper partitioning, and data locality reduces processing time. The number of tasks should be adjusted according to the limited number of cluster nodes so that the complete job stays within the **20-minute window**.

3. YARN Scheduler Policy for ETL, Reporting and ML

The cluster runs three types of jobs, so I would divide resources using **YARN queues**.

Proposed Queues

Queue	Priority	Purpose
ETL	High	Data extraction and transformation
Reporting	Medium	Reports and dashboards
Machine Learning	Normal/Medium	Training ML models

Policy

- Give **ETL jobs the highest priority** because they prepare data required by other applications.
- Reporting gets medium priority so that business reports remain available.
- ML jobs can use remaining resources and run with normal priority.
- Use **Capacity Scheduler or Fair Scheduler** to prevent one workload from taking all resources.
- Set minimum and maximum resource limits for each queue.
- When ETL workload increases, it can temporarily receive more resources.
- Use YARN's ApplicationMaster recovery and container restart mechanisms when a task fails.
- If a node fails, YARN should reschedule failed containers on healthy nodes.
- Replicated input data helps jobs continue even when one node becomes unavailable.

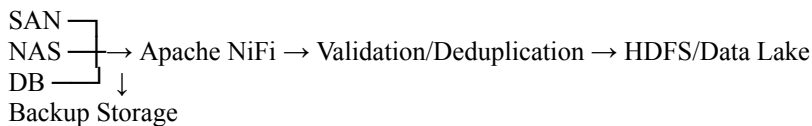
Justification:

This design provides fairness while still giving ETL higher priority. Resource limits prevent reporting or ML workloads from completely blocking other applications. Automatic task recovery improves reliability during node failures.

4. Enterprise Data Ingestion from SAN, NAS and Databases

The enterprise needs to collect data from different sources while avoiding unnecessary duplication.

Proposed architecture:



Components

- **Apache NiFi:** Used as the main ingestion and data-flow management tool.
- **SAN/NAS:** Existing enterprise storage systems can be connected to NiFi using suitable file-based ingestion methods.
- **Operational Databases:** Database connectors can capture required tables or incremental changes.
- **Deduplication:** Use file hashes, record IDs, timestamps, or transaction IDs to identify duplicate data.
- **Data Lake/HDFS:** Store the cleaned and required data in distributed storage.
- **Backup:** Keep important datasets in a separate backup location.

- **Metadata:** Maintain information about source, time, format, and destination of each dataset.
- **Incremental Ingestion:** Only new or changed database records should be transferred instead of copying complete databases repeatedly.
- **Monitoring:** NiFi provides flow monitoring and failure handling.

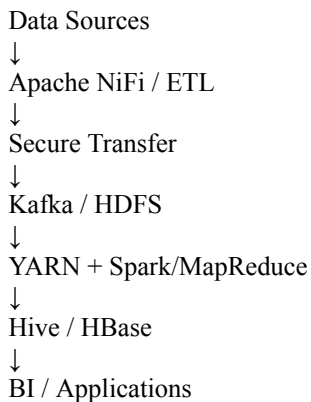
Justification:

NiFi provides a controlled way to connect different data sources. Incremental ingestion and deduplication reduce unnecessary copies. Replication and separate backups improve reliability without creating excessive duplicate datasets.

5. End-to-End Hadoop Ecosystem Solution

The solution should provide **secure data movement, high availability, and integration with Apache NiFi/ETL tools.**

Architecture



Main Components

1. Data Ingestion

- Use **Apache NiFi** to collect data from databases, files, applications, and other sources.
- ETL tools can clean and transform data before storing it.

2. Secure Data Movement

- Use **TLS/SSL** for data transmission.
- Use authentication and access control for users and services.
- Encrypt sensitive data during transfer.

3. HDFS Storage

- Store large datasets in HDFS.
- Use a replication factor of **3** for important data.
- Replication allows data to remain available when a DataNode fails.

4. High Availability

- Use **Active and Standby NameNodes**.
- Use multiple DataNodes instead of depending on a single storage server.
- YARN can restart failed tasks on healthy nodes.

5. Processing

- **YARN** manages cluster resources.
- **Spark** can be used for fast processing and ETL.
- **MapReduce** can be used for large batch-processing workloads.
- **Hive** provides SQL-based analysis.

6. Data Serving

- **Hive** can support analytical queries.
- **HBase** can be used when fast access to large structured datasets is required.

7. Monitoring and Recovery

- Monitor storage, nodes, jobs, and NiFi flows.
- Failed data transfers can be retried.
- Replicated HDFS blocks provide recovery from DataNode failures.

Overall Justification

This Hadoop architecture satisfies all three constraints:

- **Secure data movement:** NiFi with authentication and encryption protects data during transfer.
- **High availability:** HDFS replication, multiple DataNodes, and NameNode failover prevent a single failure from stopping the system.
- **NiFi/ETL integration:** NiFi manages data ingestion and transformation before the data enters HDFS and Hadoop processing.
- **Scalability:** Additional DataNodes can be added when the amount of data increases.
- **Reliability:** Replication, monitoring, retries, and backup mechanisms reduce the risk of data loss.